

UNIVERSIDADE ESTADUAL DA PARAÍBA

Projeto Unidade 1 da Cadeira Laboratório de Estrutura de Dados

Discentes: Alice Ferreira, Caio César, Sarah Oliveira

Docente: Janderson Jason

Análise de Algoritmos de Busca e Ordenação

Repositório do projeto
Vídeo de apresentação:

---

1. Introdução

Este trabalho visou implementar os algoritmos de ordenação estudados na disciplina de Estrutura de dados e comparar seus desempenhos experimentais com a complexidade assintótica conhecida esperada. A avaliação destes algoritmos será dada por meio da análise de seu desempenho com vetores em ordenação aleatória em tamanhos diferentes. Este relatório irá discutir os tempos, as concordâncias e divergências entre o desempenho dos algoritmos na teoria e na prática.

2. Metodologia

2.1 Ambiente experimental

Todo o trabalho foi conduzido em ambiente windows a partir da IDE Eclipse, e desempenhado em um notebook Asus Vivobook 15.

2.2 Algoritmos avaliados

- **Ordenação:** InsertionSort, BubbleSort (slide e otimizado), SelectionSort (slide e estável), MergeSort, QuickSort (slide e slide+shuffle), CountingSort e Arrays.sort(int[]) como referência da biblioteca padrão.

- **Busca (em vetor previamente ordenado):** Linear Iterativa, Linear Recursiva, Linear Iterativa Duas Pontas, Binária Iterativa e Binária Recursiva.

2.3 Parâmetros dos testes

- Vetores de inteiros de 1 a 100.000 com possíveis repetições;
- Tamanhos dos vetores: 10.000, 50.000, 100.000 elementos;
- Quatro cenários de ordenação: vetores aleatórios, já ordenados, em ordem inversa e parcialmente ordenados;
- Cenário parcialmente ordenado: parte-se de um vetor ordenado, sorteiam-se aproximadamente 15% das posições e embaralham-se apenas os elementos dessas posições.
- Busca pelo elemento -1, ausente do intervalo de valores, em vetores ordenados (pior caso).
- 25 execuções por combinação de algoritmo, tamanho e cenário; as 5 primeiras são descartadas como warm-up da JVM, restando 20 execuções válidas para as estatísticas.
- Tempos medidos com System.nanoTime(); reportados em milissegundos, e em algoritmos de busca foi reportado em microsegundos
- Para cada tamanho e cenário, todos os algoritmos recebem cópias da mesma entrada original; nenhum recebe como entrada a saída de outro.
- Após cada execução, o resultado é verificado (vetor ordenado; busca devolvendo -1). A verificação fica fora da medição de tempo.

3. Complexidade assintótica esperada

Algoritmo	Melhor	Médio	Pior	Estável	Mem. aux.
InsertionSort	O(n)	O(n²)	O(n²)	Sim	O(1)
BubbleSort (slide)	O(n²)	O(n²)	O(n²)	Sim	O(1)
BubbleSort (otimizado)	O(n)	O(n²)	O(n²)	Sim	O(1)
SelectionSort (slide)	O(n²)	O(n²)	O(n²)	Não	O(1)
SelectionSort (estável)	O(n²)	O(n²)	O(n²)	Sim	O(1)
MergeSort	O(n log n)	O(n log n)	O(n log n)	Sim	O(n)
QuickSort (slide)	O(n log n)	O(n log n)	O(n²)	Não	O(log n)*
QuickSort (slide+shuffle)	O(n log n)	O(n log n)	O(n²)	Não	O(log n)*
CountingSort	O(n + k)	O(n + k)	O(n + k)	Sim	O(n + k)
Busca linear (todas)	O(1)	O(n)	O(n)	–	O(1)**
Busca binária (todas)	O(1)	O(log n)	O(log n)	–	O(1)**

*Tabela – Complexidades esperadas (k = amplitude dos valores; * pilha de recursão, O(n) no pior caso; ** versões recursivas usam pilha).*

4. Resultados

4.1 Vetores aleatórios

Tempo médio de execução em milissegundos, cenário aleatório.

Algoritmo	n = 10.000	n = 50.000	n = 100.000
InsertionSort	10,9	202,7	814,8
BubbleSort (slide)	105,3	3.571	14.602
BubbleSort (otimizado)	99,8	3.509	14.312
SelectionSort (slide)	25,0	577,6	2.284
SelectionSort (estável)	25,3	606,7	2.424
CountingSort	0,799	0,635	1,113
MergeSort	1,428	6,735	12,6
QuickSort (slide)	0,584	3,977	8,509
QuickSort (slide+shuffle)	0,903	4,143	8,905
Arrays.sort (referência)	0,809	3,027	6,391

Tabela 1 – Tempos médios (ms) no cenário aleatório.

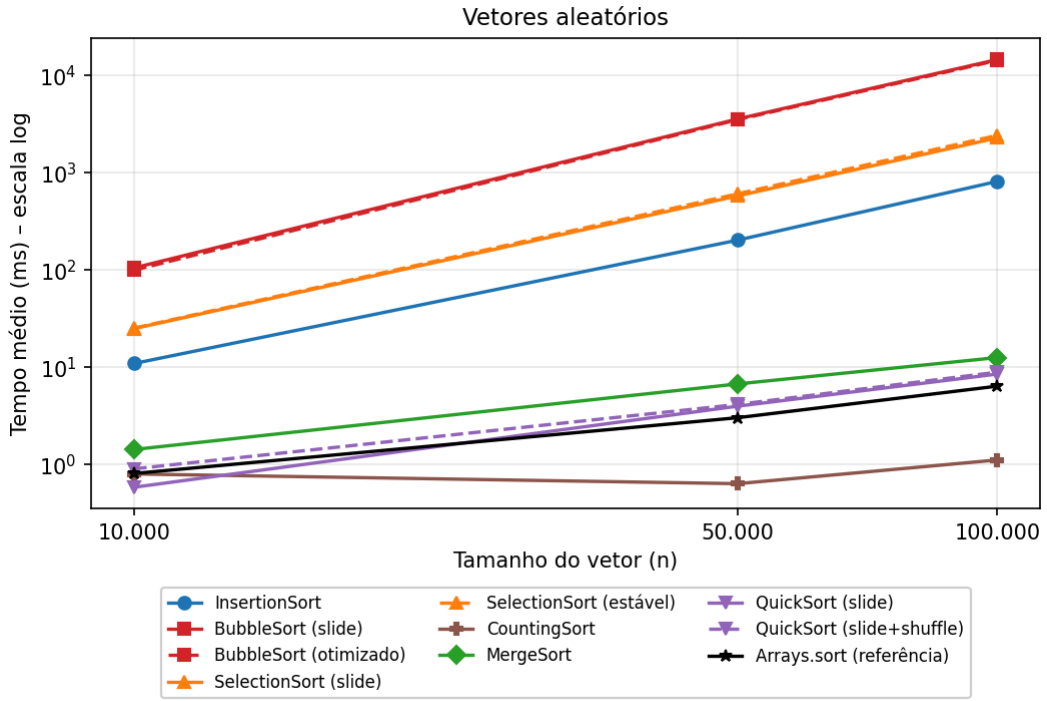


Figura 1 – Tempos de execução no cenário aleatório (escalas logarítmicas).

Os algoritmos O(n²) (Bubble, Selection e Insertion) ficam de 2 a 4 ordens de grandeza acima dos demais; no maior vetor, o BubbleSort leva cerca de 14,6 s contra 8,5 ms do QuickSort e 1,1 ms do CountingSort. As versões "otimizada" e "estável" praticamente não diferem das originais neste cenário.

4.2 Vetores já ordenados

Tempo médio de execução em milissegundos, cenário ordenado.

Algoritmo	n = 10.000	n = 50.000	n = 100.000
InsertionSort	0,019	0,068	0,159
BubbleSort (slide)	19,8	478,5	1.909
BubbleSort (otimizado)	0,008	0,048	0,153
SelectionSort (slide)	23,2	568,4	2.269
SelectionSort (estável)	22,5	562,9	2.256
CountingSort	0,286	0,481	1,007
MergeSort	0,420	2,619	5,247
QuickSort (slide)	66,6	—*	—*
QuickSort (slide+shuffle)	0,783	4,284	9,115
Arrays.sort (referência)	0,057	0,157	0,035

*Tabela 2 – Tempos médios (ms) no cenário ordenado. * Sem medição (StackOverflow na execução).*

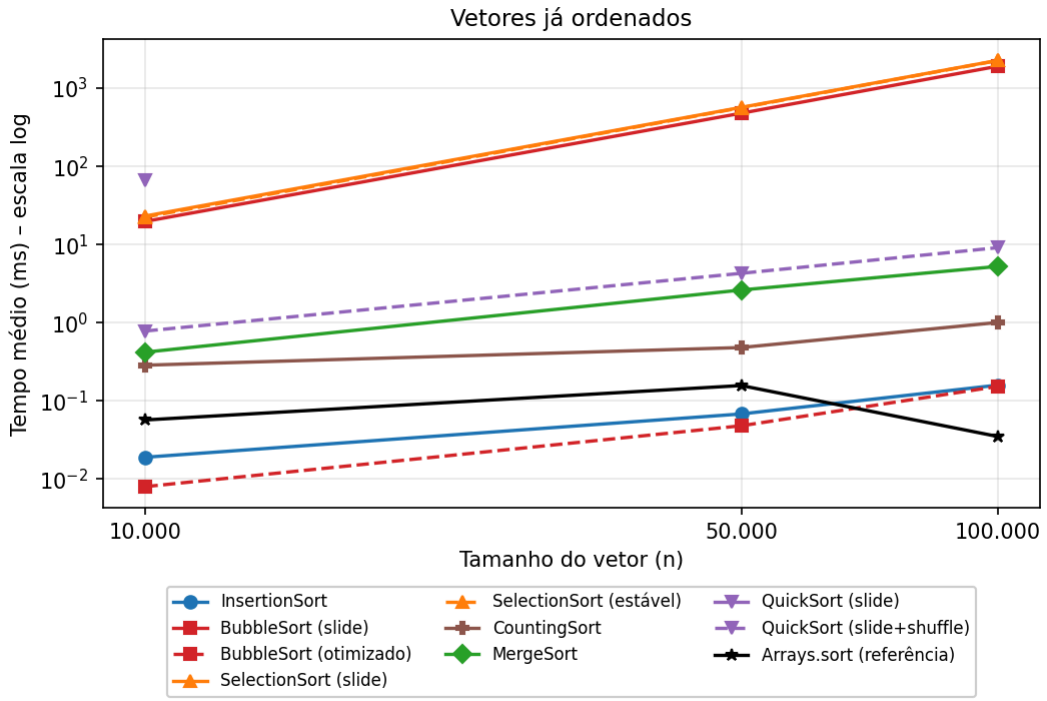


Figura 2 – Tempos de execução no cenário ordenado (escalas logarítmicas).

O InsertionSort e o BubbleSort otimizado caem para tempo linear (cerca de 0,15 ms com n = 100.000), enquanto o BubbleSort do slide e os SelectionSort continuam quadráticos. O QuickSort do slide, com pivô no último elemento, degenera para $O(n^2)$ e, em vetores grandes, esgota a pilha de recursão; a versão com shuffle não sofre esse problema.

4.3 Vetores em ordem inversa

Tempo médio de execução em milissegundos, cenário inverso.

Algoritmo	n = 10.000	n = 50.000	n = 100.000
InsertionSort	16,1	404,5	1.621
BubbleSort (slide)	80,4	2.022	8.087
BubbleSort (otimizado)	83,1	2.023	8.091
SelectionSort (slide)	36,2	1.642	10.040
SelectionSort (estável)	42,3	2.361	16.756
CountingSort	0,205	0,403	0,967
MergeSort	0,506	2,775	5,944
QuickSort (slide)	49,1	—*	—*
QuickSort (slide+shuffle)	0,744	4,194	8,694
Arrays.sort (referência)	0,092	0,259	0,100

*Tabela 3 – Tempos médios (ms) no cenário inverso. * Sem medição (StackOverflow na execução).*

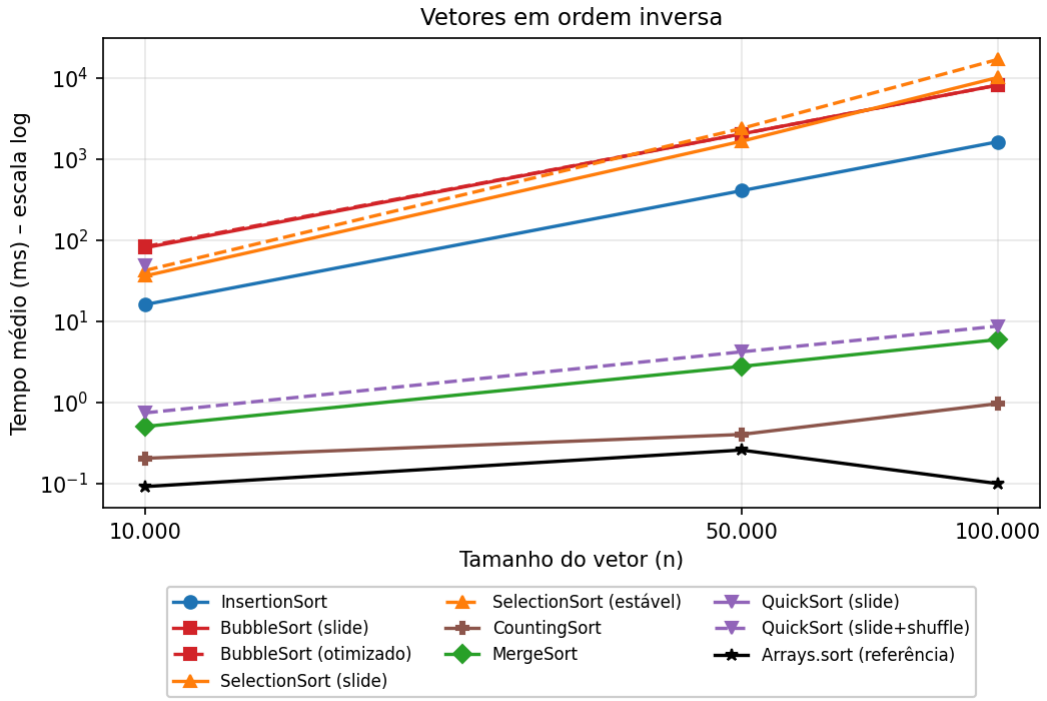


Figura 3 – Tempos de execução no cenário inverso (escalas logarítmicas).

Em algoritmos quadráticos, este cenário normalmente é o mais custoso: o SelectionSort estável chega a aproximadamente 16,8 s com n = 100.000, pois desloca elementos a cada passada. O QuickSort do slide também degenera, enquanto MergeSort, CountingSort e Arrays.sort permanecem em, no máximo, poucos milissegundos.

4.4 Vetores parcialmente ordenados (+/-15%)

Tempo médio de execução em milissegundos, cenário parcialmente ordenado.

Algoritmo	n = 10.000	n = 50.000	n = 100.000
InsertionSort	1,447	37,7	145,1
BubbleSort (slide)	36,1	1.108	4.573
BubbleSort (otimizado)	31,9	1.029	4.237
SelectionSort (slide)	22,7	566,9	2.262
SelectionSort (estável)	23,3	575,1	2.349
CountingSort	0,192	0,412	0,791
MergeSort	0,670	3,376	8,080
QuickSort (slide)	0,668	3,547	8,027
QuickSort (slide+shuffle)	0,804	4,221	9,382
Arrays.sort (referência)	0,300	1,841	3,838

Tabela 4 – Tempos médios (ms) no cenário parcialmente ordenado.

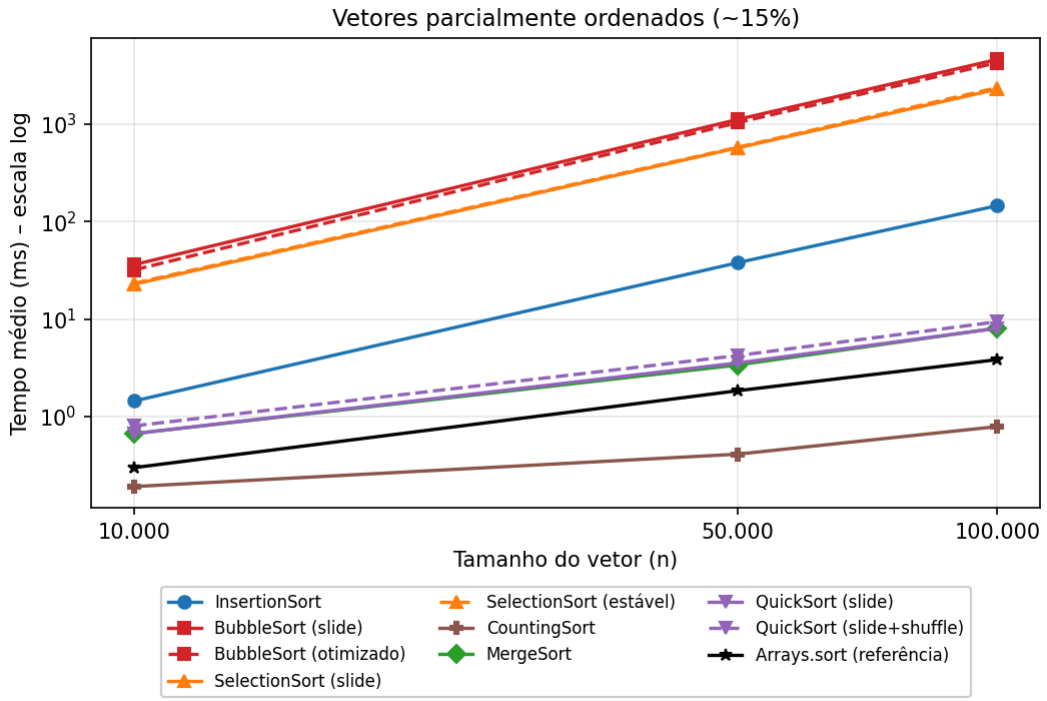


Figura 4 – Tempos de execução no cenário parcialmente ordenado (escalas logarítmicas).

O InsertionSort se beneficia da ordenação parcial (aproximadamente 145 ms com n = 100.000, contra aproximadamente 815 ms no aleatório), pois seu custo cresce com o número de inversões. O BubbleSort otimizado quase não ganha, já que com 15% das posições embaralhadas sempre há passadas com trocas.

4.5 Busca

Tempo médio da busca pelo elemento -1 em vetor ordenado, em microssegundos.

Algoritmo	n = 10.000	n = 50.000	n = 100.000
Busca Linear Iterativa	61,0	58,0	23,0
Busca Linear Recursiva	218	—*	—*
Busca Linear Duas Pontas	85,0	116	36,0
Busca Binária Iterativa	1,00	2,00	2,00
Busca Binária Recursiva	2,00	1,00	1,00

*Tabela 5 – Tempos médios (µs) da busca. * Sem medição (StackOverflow na execução).*

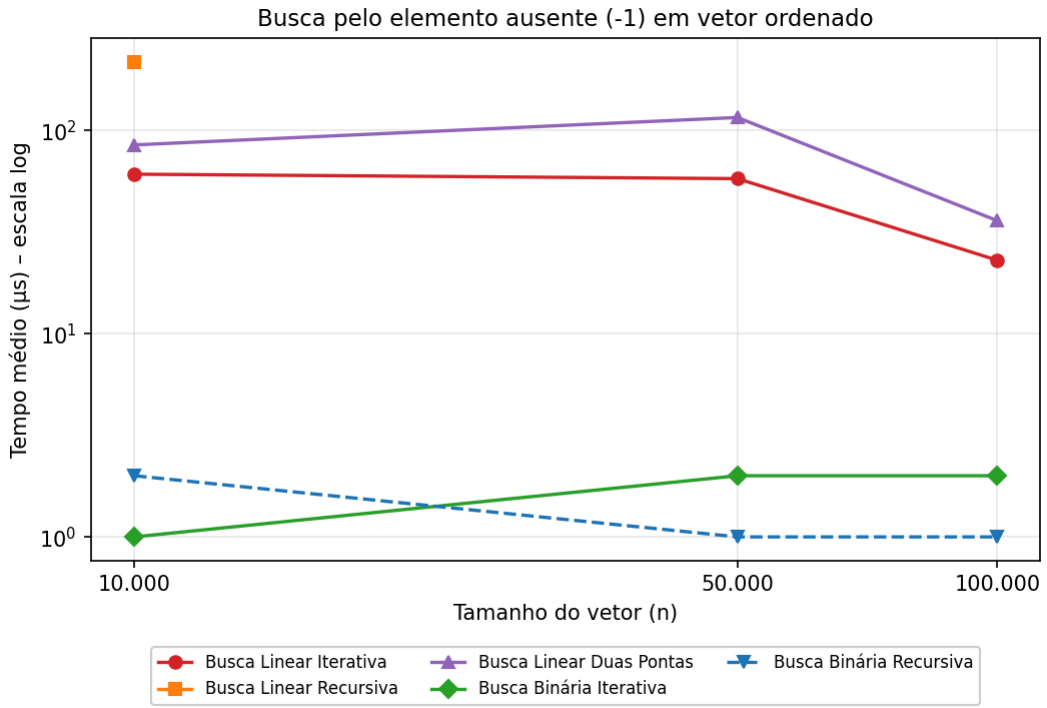


Figura 5 – Comparação entre buscas lineares e binárias (escalas logarítmicas).

A busca binária ficou na casa de 1 a 2 µs em todos os tamanhos, enquanto as buscas lineares levaram dezenas de microssegundos, como esperado de O(log n) frente a O(n). A busca linear recursiva foi cerca de 3,6 vezes mais lenta que a iterativa com n = 10.000 (218 µs contra 61 µs) e, em vetores maiores, esgotou a pilha. A versão de duas pontas não trouxe ganho: como o elemento está ausente, ela percorre o vetor inteiro e fez mais trabalho por iteração (85 µs contra 61 µs com n = 10.000).

5. Resultados e discussão

5.1 Teoria × prática

De modo geral os tempos acompanharam as complexidades esperadas. Nos algoritmos $O(n^2)$, dobrar o tamanho de 50.000 para 100.000 multiplicou o tempo por cerca de 4: BubbleSort de 3.571 para 14.602 ms (×4,1), InsertionSort de 203 para 815 ms (×4,0) e SelectionSort de 578 para 2.284 ms (×4,0). No MergeSort ($O(n \log n)$), o tempo foi de 1,43 para 6,74 e 12,59 ms; as razões observadas (4,7 e 1,9) ficaram próximas das previstas por $n \log n$ (≈ 5,9 e ≈ 2,1). O CountingSort manteve cerca de 1 ms nos três tamanhos, coerente com $O(n + k)$, em que $k = 100.000$ domina nos vetores menores.

Houve também divergências. Ao multiplicar n por 5 (de 10.000 para 50.000), o esperado para $O(n^2)$ seria multiplicar o tempo por 25, mas o BubbleSort cresceu cerca de ×34 e os demais ficaram entre ×19 e ×24, o que sugere efeitos de cache, previsão de desvios e compilação JIT. Algumas medições não foram monótonas: o CountingSort foi mais rápido com 50.000 do que com 10.000 elementos, e a busca linear iterativa não cresceu com n. Isso indica que as primeiras medições, feitas com o menor vetor, ainda sofriam influência do aquecimento da JVM, mesmo após o descarte de 5 execuções. Os tempos da busca binária (1 a 2 µs) estão próximos da resolução da medição, e por isso as diferenças entre as variantes iterativa e recursiva não são conclusivas.

5.2 Influência do cenário

As diferenças entre os algoritmos foram mais evidentes nos cenários ordenado e inverso. Com vetor ordenado ($n = 100.000$), o InsertionSort levou 0,16 ms e o BubbleSort otimizado 0,15 ms (tempo linear), contra 1.909 ms do BubbleSort do slide, cerca de 12 mil vezes mais. O SelectionSort, que sempre executa o mesmo número de comparações, foi insensível à ordem inicial, exceto no cenário inverso (≈ 10,0 s o do slide e ≈ 16,8 s o estável, contra ≈ 2,3 s no aleatório); no estável isso se explica pelos deslocamentos de elementos a cada passada, e no do slide a causa não foi investigada.

O QuickSort do slide, com pivô no último elemento, foi o mais sensível: com $n = 10.000$ levou 66,6 ms em vetor ordenado e 49,1 ms em inverso, contra 0,58 ms em aleatório (até ≈ 114 vezes mais), e com vetores maiores a recursão de profundidade n esgotou a pilha. Com o shuffle, o tempo ficou em ≈ 9 ms para $n = 100.000$ em qualquer cenário. No cenário parcialmente ordenado, o InsertionSort (≈ 145 ms) ficou entre o melhor e o pior caso, pois seu custo cresce com o número de inversões. MergeSort, CountingSort e Arrays.sort foram pouco sensíveis ao cenário; o Arrays.sort ficou extremamente rápido em vetores ordenados (0,035 ms com $n = 100.000$), provavelmente porque a implementação reconhece sequências já ordenadas.

5.3 Estabilidade

Os algoritmos estáveis são: InsertionSort, ambas as versões do BubbleSort, o SelectionSort estável, o MergeSort, por usar <= na intercalação, e o CountingSort, por preencher o vetor de saída de trás para frente. Não são estáveis o SelectionSort do slide, o QuickSort (as duas versões) e o Arrays.sort(int[]), embora para valores primitivos a estabilidade seja irrelevante, pois elementos iguais são indistinguíveis. A estabilidade importa quando se ordenam registros por mais de um critério, por exemplo ordenar alunos por nota e manter a ordem alfabética entre os de mesma nota.

5.4 Uso de memória auxiliar

O MergeSort aloca vetores auxiliares (O(n) no total, criados a cada intercalação, o que também pesa no tempo) e o CountingSort usa um vetor de contagem de tamanho k = 100.000 mais um vetor de saída de tamanho n. O QuickSort usa apenas a pilha de recursão, que é O(log n) em média mas O(n) no pior caso, como mostrou o StackOverflow nos vetores ordenados e inversos. Bubble, Selection e Insertion trabalham no próprio vetor, com O(1). Em aplicações com pouca memória ou vetores muito grandes, o consumo auxiliar pesa na escolha: o CountingSort só compensa quando a amplitude dos valores (k) é pequena em relação a n.

5.5 Escolha em uma aplicação real

Para ordenar um grande conjunto de dados, evitaríamos BubbleSort, SelectionSort e o QuickSort sem embaralhamento, que ficaram de segundos a dezenas de segundos ou falharam por estouro de pilha. O InsertionSort só faz sentido para vetores pequenos ou quase ordenados. Na prática, usaríamos Arrays.sort(): é otimizado, bem testado e foi o melhor ou ficou entre os melhores algoritmos de propósito geral em todos os cenários. Uma implementação própria se justifica quando há conhecimento específico sobre os dados, como inteiros em intervalo limitado, caso em que o CountingSort (≈ 1 ms com n = 100.000, contra ≈ 6,4 ms do Arrays.sort) foi o mais rápido, ou quando há restrições de memória ou de estabilidade que a biblioteca não atenda.

6. Conclusão

Os resultados confirmaram, em linhas gerais, as complexidades esperadas: os algoritmos $O(n^2)$ ficaram muito acima dos O(n log n), e o CountingSort foi o mais rápido por explorar o intervalo limitado dos valores. O cenário de entrada alterou bastante o desempenho de InsertionSort, BubbleSort otimizado e QuickSort do slide, mostrando que a complexidade de pior e melhor caso tem efeito real. As divergências observadas decorrem sobretudo de efeitos da JVM e do hardware. Entre a busca linear e a binária, a diferença ficou clara desde os menores vetores, reforçando a vantagem de manter os dados ordenados quando há muitas consultas.